Teoria de Software Básico: o necessário para resolver variantes da prova

Baseado nos slides **7-teoria (1 e 2)**, **8-so-red (1 e 2)** e **9-boot-so-red**. As referências do tipo `7-1 s17` indicam o arquivo e o número do slide (o número que aparece no canto do slide).

Como usar: cada seção explica o conceito como o professor o apresenta, e termina com **"Como pode cair"**, uma lista das trocas de palavras que transformam uma frase verdadeira em falsa. A prova é de V/F/NS com justificativa obrigatória nos F, mais discursivas. Por isso o objetivo é saber **o que o slide diz exatamente** e **o que muda quando se troca uma palavra**.

Onde algo **não está nos slides**, mas é necessário para responder (por exemplo, as condições de deadlock), isso vem marcado com **(fora dos slides)**.

Sumário

1. Softwares básicos
2. Arquitetura de von Neumann
3. E/S, interrupção e redução do gargalo
4. Código absoluto × relocação estática
5. Carregador absoluto
6. Montador
7. Ligador
8. Carregador de relocação
9. Ligação dinâmica e DLL
10. Sistema operacional: funções e tipos
11. Interrupções
12. Processos, escalonamento, deadlock e starvation
13. Sistema de arquivos
14. Gerência de memória real
15. Gerência de memória virtual
16. Segurança e estruturação do S.O.
17. Boot do Windows XP
18. Perguntas dos rodapés dos slides (respondidas)

19. Mapa mental: quem faz o quê
20. Como atacar um item V/F inédito

1. Softwares básicos (7-1 s3–s4)

Software de sistema (software básico):

- programas que **suportam a operação** do computador;
- são **dependentes da arquitetura** da máquina;
- permitem que o usuário **não precise entender o funcionamento interno** da máquina;
- fazem parte do conceito de **computadores virtuais**.

Software de aplicação: usa o computador como **ferramenta** para resolver um problema.

Exemplos de software de sistema e a função de cada um (decore o verbo):

Software	Função
Tradutor	traduz programas
Montador (assembler)	monta programas a partir de código Assembly
Macroprocessador	monta programas que possuem macroinstruções
Ligador (linker, binder)	**resolve referências externas** e prepara o programa para execução
Carregador (loader)	**carrega** o programa na memória
Depurador simbólico	facilita a identificação de erros
Sistema operacional	gerencia e controla a máquina

Como pode cair

- "Softwares básicos são **independentes** da arquitetura": **F**.
- Trocar as funções: "o **montador** resolve as referências externas" é F (isso é o **ligador**); "o **ligador** carrega o programa na memória" também é F (isso é o **carregador**; a exceção é o *linking loader*, que faz as duas coisas).

2. Arquitetura de von Neumann (7-1 s5–s8)

2.1 Componentes (são 4: decore a lista completa)

1. **CPU**: ULA (unidade lógica e aritmética), UC (unidade de controle), registradores e **PC** (contador de programa).
2. **Memória**: guarda **dados e programas**, na mesma memória.
3. **Periféricos de E/S**: podem ser **mapeados na memória**.
4. **Barramentos**: **dados, endereço e controle**.

> Uma frase do tipo "a arquitetura de von Neumann **é caracterizada por** CPU, memória e periféricos" está **incompleta numa lista fechada**: faltam os **barramentos**. Esse é o tipo de item que o professor marca como F.
> Outro detalhe: o slide diz "memória", não "memória RAM". Parte do código fica em ROM (o boot).

2.2 Características (7-1 s7)

- **Dados e programa ficam armazenados na memória.** Instruções e dados **só podem ser processados se estiverem na CPU**: precisam ser trazidos da memória para registradores.
- **O processamento é sequencial**, seguindo o ciclo:
  busca → decodificação → busca de operandos → execução (*fetch–execute cycle*)
  - **PC ← entry point**: quem coloca o entry point no PC é o **carregador**, ao transferir o controle para o programa.
  - **O PC é incrementado a cada ciclo de busca.**
- **Usamos linguagens imperativas porque usamos a arquitetura de von Neumann.** Isso decorre da metodologia atual: programação **modular** e **estruturada**.
- **A execução é rápida porque dados e instruções ficam em blocos de memória adjacentes → localidade de referência**.

2.3 Localidade de referência

Definição: num curto intervalo de tempo, um programa tende a concentrar seus acessos num pequeno conjunto de endereços.

- **Temporal:** o que foi acessado agora tende a ser acessado de novo em breve (laços, variáveis reutilizadas).
- **Espacial:** depois de acessar um endereço, o programa tende a acessar os vizinhos (código sequencial, vetores, matrizes, pilha).

O que causa a localidade:

- o armazenamento em blocos adjacentes, junto com a execução sequencial (o PC é incrementado), faz a próxima instrução estar "ao lado" da atual;
- laços reexecutam o mesmo trecho;
- estruturas de dados contíguas são percorridas em sequência;
- programação modular e estruturada mantém o código de cada rotina agrupado.

Por que importa: é a localidade que faz funcionarem a **cache**, os **buffers**, o **disk cache** e a **paginação com LRU**.

O que quebra a localidade: desvios (jumps, calls). Veja a pergunta do rodapé na seção 18.

2.4 Gargalo de von Neumann (7-1 s7–s8)

Há duas diferenças de velocidade:

- **velocidade da CPU ≫ velocidade de transferência da memória para a CPU**;
- **velocidade da memória ≫ velocidade de transferência dos periféricos de E/S** para a memória ou a CPU.

Como reduzir cada gargalo (esta tabela é muito cobrada; a pegadinha é trocar as linhas):

Gargalo	Técnicas de redução
CPU × memória	**memória cache**; **pipelining** (processador com múltiplos estágios)
memória × periféricos de E/S	**blocagem e bufferização**; **DMA** (acesso direto à memória); **canais** de controle de E/S

Como pode cair

- "Cache e pipelining reduzem o gargalo entre **memória e E/S**": **F**. Eles atuam em CPU × memória.
- "DMA reduz o gargalo CPU × memória": **F**. O DMA atua em memória × E/S.
- "O ciclo é busca → execução → decodificação...": **F** (ordem trocada).
- "O **montador/ligador** coloca o entry point no PC": **F**. Quem faz isso é o **carregador**.
- "O PC é **decrementado**" ou "é incrementado na execução": **F**. Ele é incrementado **a cada ciclo de busca**.
- "Linguagens **funcionais** são usadas por causa de von Neumann": **F**. O slide fala em linguagens **imperativas**.
- "Instruções podem ser processadas diretamente na memória": **F**. Elas só são processadas **na CPU**.

3. E/S, interrupção e redução do gargalo (7-1 s9–s12)

3.1 Duas formas de resolver uma operação de E/S (7-1 s9)

1. **Laço de espera ocupado** (*busy waiting*): a CPU fica perguntando "a E/S terminou?" e **desperdiça CPU**. É antigo e não se usa mais (8-1 s11).
2. **Interrupção.** A sequência é esta (decore, porque é o roteiro de um **page fault** e de qualquer leitura de disco):
    i. A E/S gera uma **chamada aos serviços do S.O.**
    ii. Se a informação **não está disponível no buffer**, o **processo é bloqueado** numa fila específica.
    iii. Gera-se uma **chamada ao supervisor**, que **cria o programa para o canal de E/S** e as controladoras.
    iv. A operação é realizada no periférico adequado.
    v. **O canal gera uma interrupção.**
    vi. **O controlador de interrupções desbloqueia o processo.**
    vii. **O processo é colocado na fila de "prontos".**

> **Quem tira o processo de "bloqueado" e o põe em "pronto"?** A **interrupção**, tratada pelo controlador de interrupções.
> **Quem tira o processo de "pronto" e o põe em "executando"?** O **escalonador**.
> Trocar esses dois é uma pegadinha clássica.

3.2 O disco é o fator limitante (7-1 s10–s11)

Ordens de grandeza: a memória e a CPU operam em torno de **10^{-9} s**, a LAN (100 Mbit/s) em torno de **10^{-8} s** e o **disco** em torno de **10^{-3} s**.

Técnicas para lidar com isso:

- **Multiprogramação:** o processo que precisa do disco é **bloqueado**, e a CPU é dividida entre os outros. Quando a interrupção chega, ele **acorda** e volta para a fila de **prontos**.
- **RAM disk:** parte da memória **simula um disco** (sem a lentidão e **sem a não volatilidade**, ou seja, é **volátil**). Costuma simular disco flexível.
- **Disk cache:** um bloco de memória guarda **páginas do disco**; antes de ir ao disco, o sistema procura no cache.
- **Striping:** o arquivo é dividido em **fatias**, cada uma num disco diferente, e as fatias são transferidas em paralelo. **O RAID usa striping.**

3.3 Buffers (7-1 s12)

- **Buffer:** trecho de memória do tamanho de um ou mais **setores ou blocos** (registros físicos). O objetivo é **reduzir os acessos ao disco**.
- O sistema de E/S mantém **pelo menos um buffer por arquivo aberto** (em mainframes, vários por arquivo).
- **Gargalo de buffer:** com **um só** buffer, a CPU precisa esperar ele encher para processar, e depois esperar outra leitura física.
- **Com 2 ou mais buffers (*double buffering*):** enquanto o S.O. processa um buffer, o processador de E/S **enche o outro**. Isso cria paralelismo entre processamento e E/S.

Como pode cair

- "Com um único buffer por arquivo a leitura sequencial é eficiente": **F**. O slide diz que um só buffer é **moroso**.
- "RAM disk é não volátil": **F**.
- "O escalonador desbloqueia o processo ao fim da E/S": **F**. Quem faz isso é a **interrupção** (controlador de interrupções).
- "O laço de espera ocupado economiza CPU": **F**. Ele **queima** CPU.

4. Código absoluto × relocação estática (7-1 s13–s14; 7-2 s38–s39)

4.1 Código absoluto

- Ocupa **posições fixas e reais** de memória e usa **endereços fixos e reais**.
- **Ainda é usado:** na **ROM, nos procedimentos de boot**. No endereço **FFFF:0000 da ROM BIOS** há uma instrução **jmp** para o código a ser executado.
- Exige **conhecer de antemão a ocupação da memória** (7-2 s36).
- Os primeiros tradutores geravam código absoluto: era preciso traduzir **todos os módulos ao mesmo tempo**, e as bibliotecas eram distribuídas como **fonte**.

4.2 Código relocável e relocação estática

Muitas vezes **não é prático saber onde o programa será carregado**. Por isso o tradutor marca **quais partes do programa precisam ser modificadas** na carga. Um programa objeto com essa informação é um **programa relocável**.

- Instruções com **endereçamento relativo** (relativas ao PC, por exemplo) **não precisam ser modificadas**.

Relocação estática:

- o código pode ser executado **em qualquer posição** da memória;
- usa **endereços relativos a uma base β** (normalmente 0);
- **endereço efetivo (absoluto) = offset + α**, onde α é o endereço de carga dado pelo S.O.;
- a relocação **soma a constante de relocação k** aos endereços relativos;
- é feita **antes da execução**, em tempo de tradução, ligação ou carga. **Não acontece durante a execução.** Ajustar endereços durante a execução é **relocação dinâmica**, feita com registrador de base ou paginação (seção 14).
- o código é carregado em **posições contíguas** a partir de α.

> **A relocação estática viabiliza a construção de bibliotecas de código objeto** (7-1 s14, s23): uma biblioteca compilada uma vez pode ser ligada em qualquer programa e em qualquer posição.

Exemplo:

```
Montado com base 0:
  offset 10:  JMP 40     ; 40 é endereço relocável (rótulo LOOP)
  offset 40:  LOOP: ...
Lista de endereços relocáveis: {11}   ; o campo de endereço do JMP

Carregado em α = 1000:
  endereço 1010:  JMP 1040   ; 40 + 1000
  endereço 1040:  LOOP: ...
```

Se o JMP fosse **relativo ao PC** (por exemplo, "pule 30 posições"), ele **não** entraria na lista e não seria alterado.

Como pode cair

- "Código absoluto **não é mais usado**": **F**. Ele é usado no boot (ROM BIOS e carregador absoluto).
- "A relocação estática é feita durante a execução": **F**. Isso é relocação **dinâmica**.
- "Instruções com endereçamento relativo precisam ser relocadas": **F**.
- "Código relocável permite bibliotecas de código objeto, **como o bytecode Java**": **F**. A primeira parte é verdadeira, mas o exemplo está errado: bytecode é código de **máquina virtual**, e sua portabilidade vem da **JVM**, não da relocação.

- "Endereço absoluto = offset × α": **F**. É uma **soma**.

5. Carregador absoluto (7-1 s15–s16)

- **Inicia a carga do sistema operacional.** É o primeiro programa a rodar.
- É **código absoluto**, armazenado em **memória não volátil** (ROM, flash), em **posição fixa e conhecida**.
- **Ativação:** quando o operador aperta o botão (liga/desliga ou um botão específico).
- **No mainframe** ele se chama **IPL** (*Initial Program Load*).

Versão mínima (*pressload*): lê 80 palavras para `memory[0..79]` e transfere o controle para `memory[0]`.

Algoritmo completo:

```
Load: lê um registro para o buffer r
      LA := memory[r]        ; campo 1: endereço de carga (load address)
      n  := memory[r+1]      ; campo 2: nº de palavras/bytes deste registro
      if n = 0 then transferto(LA)       ; registro final: LA é o entry point
      copia n palavras de memory[r+2..] para memory[LA..]   ; campo 3: dados
      goto Load
```

Layout do registro: endereço de carga | número de palavras | informação a carregar.

> Ele **não reloca nada**: copia os dados exatamente para os endereços indicados. Por isso é "absoluto".

Como pode cair

- "Fica em memória **volátil**" ou "em RAM": **F**. Fica em memória **não volátil** (ROM/flash).
- "Reloca o código carregado": **F**. Isso é o **carregador de relocação**.
- "Em mainframe é chamado de IPL": **V**.
- "É ativado pelo sistema operacional": **F**. Ele **carrega** o S.O. e é ativado pelo **operador** (botão).

6. Montador (7-1 s17–s31)

6.1 Definição, entrada e saída (7-1 s17)

O montador **converte um programa Assembly em código objeto relocável** (código de máquina), **se o programa Assembly estiver correto**.

- **Entrada:** arquivo **texto** com código Assembly.
- **Saída:**
  - arquivo **binário com código de máquina relocável**;
  - **tabela (ou lista) de endereços relocáveis**;
  - **lista de referências externas**, ou seja, símbolos que o montador não consegue resolver.
- **Funções básicas:**
  - verificação de sintaxe;
  - resolução de endereços simbólicos;
  - tradução de mnemônico em código de máquina;
  - tratamento de macros.
- **Se houver erro de sintaxe, o montador não gera código** e emite uma mensagem de erro.

6.2 Estruturação física: número de passos (7-1 s18)

"Passo" é **uma leitura do código-fonte**.

Passo único	Múltiplos passos
Lê o fonte **uma só vez**.	**Pré-passo:** trata macros e diretivas.
Monta a tabela de símbolos, verifica a sintaxe, gera o objeto relocável e expande macros, tudo junto.	**Passo 1:** monta a tabela de símbolos, verifica sintaxe e semântica, gera o código intermediário.
Mais complexo, por causa da **referência futura**.	**Passo n:** gera o código intermediário **otimizado**.
	Passo final: gera o **código objeto relocável**.

Referência futura: um símbolo é **usado antes de ser definido**. Exemplo: `JMP FIM` aparece antes de `FIM:` . No passo único, o endereço ainda é desconhecido quando o montador precisa gerar a instrução. Veja como resolver isso na seção 18.

6.3 Estruturas de dados do montador (7-1 s19–s24)

Estrutura	Para que serve
Contador de programa (PC do montador)	**Simula a alocação de memória.** Começa em 0 ou no valor da diretiva `org` . Depois de cada sentença, soma o tamanho da instrução ou do dado. Quando um símbolo é definido, o **valor atual do PC vira o endereço do símbolo**. Representa um **offset em relação a uma base** (sugestão: inteiro longo).
Tabela de símbolos (TS)	Guarda símbolos **internos e externos** (variáveis, procedimentos, rótulos, constantes). Campos: identificador, tipo, valor (endereço), valor inicial, local × externo, declarado × referenciado. Sugestão: **árvore binária de pesquisa**. Alguns montadores incluem os **nomes dos registradores** na TS.
Tabela de mnemônicos	Sintaxe e código de operação de cada mnemônico: mnemônico, tipo (instrução ou **pseudoinstrução**), rótulo (obrigatório, opcional ou proibido), nº e tipos de operandos, opcode e nº de bytes. Sugestão: **vetor ordenado** com ponteiros para funções "verifica sintaxe".
Lista de referências externas	Símbolos **referenciados mas não declarados** no programa, que são externos. É uma **lista encadeada reversa** usada pelo **ligador**. Alternativa: um endereço local para cada externo (o **vetor de transferência**).
Lista de endereços relocáveis	Posições que dependem da base. É usada pelo **ligador e pelo carregador de relocação**. **É a relocação que possibilita bibliotecas de código objeto.**
Tabela de macros	Nome, nº de parâmetros e ponteiro para o texto da macro.
Outras	Tabela de mensagens de erro, indicador de erro, indicador de símbolo referenciado e não declarado.

6.4 Estrutura lógica (7-1 s25–s31)

```
Fonte → [Pré-passo] → Análise léxica → tokens → Análise sintática → árvore
      → Análise semântica → código intermediário
      → Síntese: Otimização (opcional) → Geração de código → Código objeto RELOCÁVEL
```

A **análise** é composta pelas etapas léxica, sintática e semântica. A **síntese** é composta pela otimização e pela geração de código.

- **Pré-passo:** executa diretivas que **alteram o fonte**: montagem condicional, inclusão de arquivos e substituição de macros (com e sem parâmetros).
  - **Saída:** fonte "limpo", sem comentários e com as macros expandidas.
  - **Parâmetros de macro não entram na TS**, porque não têm tipo.
- **Análise léxica:**
  - agrupa os caracteres em **tokens** (identificadores, delimitadores, operadores, números, palavras-chave);
  - anexa a cada token um rótulo de tipo;
  - **converte números para binário**;
  - insere identificadores na TS.
  - Lembre que o fonte é só uma sequência de caracteres, estruturada apenas pelas quebras de linha.
- **Análise sintática:** identifica as **unidades sintáticas** (comandos e declarações), verifica se formam uma **estrutura válida** e chama o semântico.
- **Análise semântica:**
  - verifica se a estrutura é válida **dentro do contexto**;
  - **mantém a TS**;
  - insere informações implícitas (*defaults*). Exemplo: em `MOV AX,10h` , a constante é armazenada como **word**;
  - **gera o código intermediário**. A geração é **interrompida se houver erro**.
- **Síntese:**
  - **otimiza o código intermediário (opcional)**. **A otimização é do código, não das funções implementadas pelo programador**: ela não troca o seu algoritmo por outro melhor;
  - **gera código objeto relocável**;
  - **só é executada se não houver erros na análise.**

Como pode cair

- "O montador gera código **absoluto**" ou "**executável**": **F**. Ele gera código objeto **relocável**.
- "A síntese gera código absoluto": **F**.
- "O montador resolve as referências externas": **F**. Ele só as **lista**; quem resolve é o **ligador**.
- "Com erro de sintaxe, o montador gera o código e avisa": **F**. Ele **não gera** o código.
- "A síntese faz otimizações **algorítmicas**": **F**. A otimização é **do código intermediário**.
- "A otimização é obrigatória": **F**. É **opcional**.
- "O passo único é mais simples": **F**. Ele é **mais complexo** por causa das referências futuras.
- "A análise léxica verifica se os comandos formam uma estrutura válida": **F**. Isso é a **sintática**.

- "Parâmetros de macro são inseridos na TS": **F**.
- "A síntese é executada mesmo com erros na análise": **F**.

7. Ligador (7-2 s34–s48)

7.1 Definição (7-2 s34)

O ligador é o **programa que resolve as referências externas** de um programa.

- **Entrada:** arquivos **objeto com código relocável**: os módulos do usuário e as **bibliotecas** objeto (7-2 s37).
- **Saída:** arquivo **binário com código de máquina relocável executável**, mais os **endereços relocáveis**.
- **Funções básicas: ligação de módulos** e **resolução de referências externas**.

> A saída é **executável e relocável ao mesmo tempo**. É executável porque não tem mais referências externas pendentes. É relocável porque o carregador ainda vai somar α na carga.

Exemplo do slide (7-2 s35): o módulo principal declara `extrn sub1:far` e faz `call sub1` . O outro módulo define `sub1 proc far` e declara `public sub1` .

- `extrn` = **símbolo externo**: usado aqui, definido em outro módulo.
- `public` = **símbolo interno, exportado**: definido aqui, pode ser usado por outros.

7.2 Símbolos (7-2 s40)

- **Símbolos internos (públicos):** definidos em P, que **podem ser referenciados por outros** módulos.
- **Símbolos externos: referenciados em P mas definidos em outro módulo.**

7.3 Etapas da ligação (7-2 s40), nesta ordem

1. **Ler e relocar o módulo objeto P.**
2. **Atualizar a tabela global de símbolos** com os **símbolos internos (públicos)** de P.
3. **Concatenar P aos módulos anteriores.**
4. **Resolver as referências externas de P.**

Divisão lógica (7-2 s41): (a) relocação de todo o programa objeto; (b) resolução das referências externas.

7.4 Número de passos (7-2 s41–s43)

Aqui, "passo" é uma **leitura do arquivo objeto**.

- **Um passo: mais complexo**, porque precisa resolver referências externas **antes de ter a tabela global de símbolos completa**. É o mesmo problema da referência futura no montador.
- **Dois passos:**
  - **Passo 1, relocação:** lê P, **calcula k e reloca P**; coloca os **símbolos públicos** de P na tabela global; **verifica a validade do entry point**.
  - **Passo 2, resolução:** obtém as referências externas de P e as **resolve consultando a tabela global**.

Cálculo de k (7-2 s43): cada módulo foi montado com base 0. Ao concatenar:

```
P1: k = 0
P2: k = size(P1)
P3: k = size(P1) + size(P2)
```

Exemplo: P1 tem 100 bytes, P2 tem 50 e P3 tem 80. Então $k_1 = 0$, $k_2 = 100$ e $k_3 = 150$. Se `sub1` é pública em P2 no offset 20, na tabela global ela vale **120**. Toda chamada `call sub1` em P1 ou P3 recebe 120, que continua relativo ao início do programa ligado. Na carga, o carregador soma α.

7.5 Estruturas de dados (7-2 s44–s47)

- **Código objeto:** organizado em **seções com registros de tamanho variável**. O formato **depende da arquitetura e de quem desenvolve o S.O.** (o formato da Microsoft tem registros que o da Intel não tem).
  - **FIXUPP** é o registro que contém a **tabela de endereços relocáveis, incluindo os externos**.
  - `gcc -c` gera o `.o`. Por padrão, o ligador roda logo depois do compilador e o `.o` é apagado.
- **Lista de referências externas:** os símbolos referenciados mas não declarados.
- **Vetor de transferência** (abordagem alternativa): associa a cada externo **um endereço local** onde será armazenado o endereço real do símbolo, e o código acessa o símbolo com **endereçamento indireto**.
  - **A ligação fica rápida**: só é preciso escrever o endereço uma vez, no vetor.

    - **A execução fica mais lenta**: cada acesso ao externo exige **2 acessos à memória** em vez de 1.
- **Tabela de símbolos global:** guarda os **símbolos públicos** de cada módulo. Campos: identificador, tipo, valor (endereço) e valor inicial. Sugestão: árvore binária de pesquisa.

7.6 Tipos de ligador (7-2 s48, s50)

Linkage editor	Linking loader (ou *binder*)
Faz a ligação **antes da carga**.	Liga e **em seguida carrega** na memória.
Grava o resultado num arquivo, chamado **módulo de carga**.	**Não gera arquivo executável.**
Na carga, só é preciso **relocar**: a única alteração é a dos endereços relativos.	Em geral tem 2 passos: (1) atribui endereços aos externos; (2) faz a carga, a relocação e a ligação.
Mais eficiente se o programa vai ser executado várias vezes. Na manutenção, não é preciso recompilar tudo.	O slide diz: "se o programa precisa ser alterado com frequência, o uso de um linkage editor é mais eficiente".

Em grande porte existem três tipos de módulo: **fonte, objeto e de carga**.

O terceiro tipo é a ligação dinâmica (*dynamic linking*, seção 9): a ligação acontece **em tempo de execução**.

Como pode cair

- "O arquivo gerado pelo ligador **não é executável**": **F**.
- "O ligador gera código absoluto": **F**. O código continua **relocável**.
- "A tabela global guarda os símbolos **externos**": **F**. Ela guarda os **públicos/internos**.
- Etapas fora de ordem, por exemplo resolver antes de relocar: **F**.
- "O ligador de um passo é mais simples": **F**.
- "O linking loader gera um módulo de carga reutilizável": **F**. Isso é o **linkage editor**.
- "O vetor de transferência deixa a execução mais rápida": **F**. Ele deixa a **ligação** mais rápida e a **execução** mais lenta (dois acessos).
- "A relocação feita pelo ligador é dinâmica": **F**. É **estática** (k = soma dos tamanhos).
- "Os tipos de ligador são linkage editor, linking loader e ligação dinâmica (DLL)": **V**. É exatamente a lista dos slides "Tipos de Ligadores". "De um passo" e "de dois passos" é **outra classificação** (por número de leituras), não outro tipo.

8. Carregador de relocação (7-2 s49)

Carrega na **RAM** um programa **executável que está em disco** e o **reloca** para o endereço de carga fornecido pelo S.O.

Algoritmo (ambiente monousuário):

1. **Lê o tamanho do programa e o entry point.**
2. **Pede ao gerente de memória um bloco contíguo** com tamanho suficiente para o programa inteiro.
3. **Carrega o programa a partir do endereço dado pelo S.O. (α), relocando os endereços relocáveis**: cada um recebe + α.
4. **Transfere o controle para o programa, a partir do entry point** (PC ← α + entry point).

O problema desse esquema (pergunta do slide) está respondido na seção 18.

Comparação entre os carregadores

	Carregador absoluto	**Carregador de relocação**	**Linking loader**	**Carga dinâmica (DLL)**
O que carrega	o início do S.O. (boot)	um executável relocável (saída do linkage editor)	módulos objeto, ligando-os	rotinas de biblioteca, sob demanda
Quando reloca	nunca (endereços fixos)	**na carga**	**na carga**, junto com a ligação	**em execução** (fixup)
Onde fica / quem ativa	ROM, ativado pelo botão	disco, ativado pelo S.O.	disco, ativado pelo S.O.	S.O., na primeira chamada

Como pode cair

- "Aloca blocos **não contíguos**": **F**. O bloco é **contíguo**.
- "Reloca durante a execução": **F**. Reloca **na carga**.
- "Quem fornece o endereço de carga é o próprio programa": **F**. Quem fornece é o **S.O.**
- "Transfere o controle para o primeiro byte do programa": **F**. Transfere para o **entry point**.

- Uma frase que só omite "e o entry point" no passo 1, sem afirmar nada errado, tende a ser **V**: omitir um detalhe de um processo não torna a frase falsa (veja a seção 20).

9. Ligação dinâmica e DLL (7-2 s50 em diante)

- A **ligação ocorre em tempo de execução.**
- A rotina é **carregada e ligada quando é chamada pela primeira vez.**
- **Vários programas compartilham uma mesma cópia** da rotina na memória.
- Rotinas só são carregadas **se forem realmente necessárias.**
- A carga é feita **em colaboração com o S.O.**

Biblioteca dinâmica × estática:

- O código da biblioteca dinâmica **não é incluído no arquivo objeto**: ele fica em arquivo separado no disco.
- O ligador **só registra** quais bibliotecas e funções são usadas. O código é acessado depois, **pelo carregador, na carga ou na execução**.
- **Quem resolve a relocação das funções de biblioteca é o carregador**, usando uma tabela de relocação chamada **fixup**, que é **atualizada em tempo de execução**.

Nomes dos arquivos:

- **Windows: DLL** (*Dynamic Link Library*), com extensões `.dll`, `.ocx` (ActiveX), `.drv` (drivers legados), `.cpl` (painel de controle) e, mais raramente, `.icl`, `.fon` e `.fot` (ícones e fontes).
- **Linux e Solaris:** `lib*.so` (*shared object*), opcionalmente seguido do número de versão.

Duas formas de carregar uma DLL:

1. **Normal:** todas as DLLs do programa são carregadas **quando o programa é carregado**. Se o S.O. não achar uma delas, **o programa nem é carregado**.
2. **Delayed loading:** a DLL só é carregada **na primeira chamada** a uma de suas funções. **Vantagem: permite tratar exceções**, porque se a DLL faltar o S.O. retorna uma exceção durante a execução.

Vantagens das DLLs:

- **compartilhamento** de uma única cópia das funções entre vários programas;
- arquitetura **modular**;

- suporte a programas **multilinguagem**: linguagens diferentes podem chamar a mesma DLL, desde que chamem a função do mesmo jeito;
- facilita **versões internacionais**: as strings de cada língua ficam numa DLL separada.

Exemplo do slide: na DLL, a função é declarada com `__declspec(dllexport)`. No programa principal, com `__declspec(dllimport)`.

Como pode cair

- "Na ligação dinâmica, o código da biblioteca é copiado para o executável": **F**. Isso é a ligação **estática**.
- "No carregamento normal, se a DLL faltar, o programa roda e gera exceção": **F**. Nesse caso o programa **nem carrega**. A exceção acontece no **delayed loading**.
- "Cada programa recebe sua própria cópia da rotina": **F**. A cópia é **compartilhada**.
- "Quem atualiza o fixup em execução é o ligador estático": **F**. Quem atualiza é o **carregador**.

10. Sistema operacional: funções e tipos (8-1 s3–s5)

S.O.: software que **gerencia os recursos** do computador e **provê uma interface amigável** entre o hardware/firmware e o usuário.

Funções básicas:

- **Facilitar o uso do computador.**
- **Prover interface ao usuário:** linguagem de controle (orientada a caracteres) ou janelas e menus.
- **Prover serviços aos programas**, por meio de uma interface de programação, e um **ambiente em tempo de execução**.
- **Gerenciar os recursos:**
  - interrupções;
  - escalonamento;
  - memória real;
  - memória virtual;
  - rede;
  - segurança e recuperação de erros.

Tipos de S.O. e exemplos (8-1 s4):

Tipo	Ideia	Exemplos
Single-job	um único usuário e um único job	CP/M, DOS 1.0
Multiprogramado	diversos usuários e programas na memória	IBM VM
Multiprocessado	diversos processadores	SUN Solaris, IBM z/OS
Rede	serviços de rede; **a rede não é transparente**	Novell NetWare
Distribuído	**a rede é transparente** ao usuário	Amoeba, NCE Tropix, K42
Batch	**processamento em lote**, sem interação	IBM MVS
Time-sharing	**uso de time slice**, interativo	IBM TSO
Tempo real	prazos rígidos	QNX, FreeRTOS, TI-RTOS

Batch × time sharing (os slides não trazem tabela; esta é a leitura usada nas provas):

	Batch	Time sharing
Interação	nenhuma: jobs em lote	interativo, com usuário no terminal
Perfil típico	**CPU intensivo**	**E/S intensivo**, transações curtas
Métrica principal	**throughput**, turnaround	**tempo de resposta**
Escalonamento	pode ser não preemptivo (FIFO, prioridade)	**preemptivo**, com time slice / **round-robin**
Memória real típica	**partições**	**swapping**: troca de processos via interrupção de time slice (8-2 s18)

Máquina estendida (8-1 s5): as rotinas do ambiente de execução definem uma **máquina estendida**, que é uma **máquina virtual**: **cada programa tem a ilusão de ter sua própria máquina**. O IBM VM suporta milhares de máquinas virtuais.

Como pode cair

- "Time sharing é adequado a transações **CPU intensivas**": **F**. As transações são **interativas e E/S intensivas**.
- "S.O. de rede torna a rede transparente": **F**. Quem torna a rede transparente é o S.O. **distribuído**.

- Trocar os exemplos (por exemplo, "MVS é time-sharing"): **F**. O MVS é **batch** e o TSO é **time-sharing**.

11. Interrupções (8-1 s6–s8)

11.1 Conceito

- **Interrupção é um sinal que altera o fluxo normal de execução de um programa.** Ela **transfere automaticamente o controle** para uma **rotina de tratamento**, que normalmente faz parte do S.O. Os usuários **podem definir suas próprias rotinas**.
- Depois da rotina, o controle **volta para o programa**.
- **Pode ser assíncrona** em relação ao programa.
- Quando ocorre, **o estado da CPU é salvo**: os registradores de estado guardam o **contexto do processo interrompido** (a **PSW**).
- **Cada interrupção tem um nível de prioridade**, definido pela CPU ou por um dispositivo externo.

11.2 Fontes de interrupção

- **Software:**
  - instrução **SVC** (mainframe) ou **INT** (80x86);
  - **condição detectada na execução**: on condition, **divisão por zero**, `try...catch` .
- **Dispositivo de hardware.**

11.3 Modo usuário × modo supervisor

- Os serviços podem ser pedidos por **endereços fixos** (interrupções de hardware) ou por **interrupções de software** (SVC/INT).
- **A interrupção comuta a máquina do modo usuário para o modo supervisor.**
- **Em modo supervisor, todas as instruções podem ser executadas.** As que **só** podem ser executadas em modo supervisor são as **instruções privilegiadas**.

11.4 Reconhecimento da interrupção de hardware (8-1 s8)

1. Quando possível, a CPU testa se há pedido de interrupção **mascarável**: o pino **INTR** está ativo e o flag **IF** está habilitado.

2. Se houver, a CPU propaga o sinal **alto (1) na linha de ACK**, que passa por todas as fontes **em ordem decrescente de prioridade**. É uma cadeia, o *daisy chain*.
3. A fonte que quer interromper **abaixa o sinal de ACK** (as de menor prioridade não o recebem) e **coloca sua identificação no barramento de dados**. Esse esquema **permite interrupções em cascata**.
4. A CPU usa a identificação como **índice no vetor de interrupções**, que contém **ponteiros para as rotinas de tratamento**. **A localização do vetor é controlada pelo S.O.**

Como pode cair

- "A interrupção comuta do modo supervisor para o modo usuário": **F**. A comutação é **de usuário para supervisor**.
- "Instruções privilegiadas podem ser executadas em modo usuário": **F**.
- "Interrupções são sempre síncronas": **F**. Elas **podem ser assíncronas**.
- "Divisão por zero não gera interrupção": **F**. É uma interrupção de software, por condição detectada na execução.
- "O vetor de interrupções contém o código das rotinas": **F**. Ele contém **ponteiros** para as rotinas.
- "A fonte é reconhecida percorrendo as fontes em ordem crescente de prioridade": **F**. A ordem é **decrescente**.

12. Processos, escalonamento, deadlock e starvation (8-1 s9–s10)

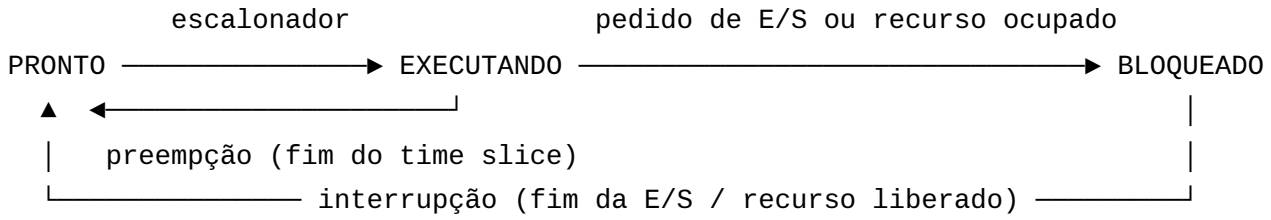
12.1 Processos e escalonamento

- **Processo é um programa em execução.** Vários processos competem pela CPU. **Com uma única CPU, só um processo executa por vez.**
- **O escalonador define qual processo executa** naquele momento.
- **Estados:** pronto, executando, bloqueado, finalizado...
- Quando o processo sai de execução, suas informações ficam guardadas no **PCB** (*Process Control Block*, bloco de controle do processo).
- **Algoritmos:** time slice, **round-robin**, prioridades.
- **O escalonamento pode ser preemptivo ou não preemptivo.**
- **Objetivo: maximizar o throughput** e **minimizar o turnaround time e o response time**.

Definições (fora dos slides, mas necessárias):

- **Throughput:** número de processos concluídos por unidade de tempo.
- **Turnaround time:** tempo total entre a submissão e o término do processo.
- **Response time:** tempo até a **primeira resposta**. É o que importa em sistemas interativos.
- **Preemptivo:** o S.O. pode tirar a CPU do processo, por fim do time slice ou porque chegou um processo de prioridade maior.
- **Não preemptivo:** o processo só libera a CPU quando termina ou se bloqueia.
- **Round-robin:** fila circular. Cada processo recebe um **time slice**; se não terminar, volta para o fim da fila de prontos. É **justo** e **evita starvation**.
- **Prioridade:** sempre roda o de maior prioridade. **Pode causar starvation** dos processos de baixa prioridade. A solução é o **envelhecimento** (*aging*): a prioridade aumenta com o tempo de espera.

Transições de estado:



12.2 Alocação de recursos, região crítica e deadlock (8-1 s10)

O S.O. controla a alocação de recursos definidos pelo usuário assim: quando um processo pede um recurso, o S.O. **verifica se ele já está com outro processo**.

- **Se estiver alocado → o processo é bloqueado.**
- **Se estiver livre → o recurso é alocado** e o controle volta para o processo.
- No slide, esse mecanismo aparece com o rótulo **"Região crítica"**.

Também **podem ser implementados mecanismos para evitar deadlock.** O rodapé do slide diz: *"O deadlock caracteriza uma situação de contenda em que ninguém vence."*

Região crítica:

- No slide, região crítica é o mecanismo em que, se o recurso está ocupado, quem chega fica bloqueado. Ou seja, região crítica está ligada a **garantir a exclusão mútua**.
- Nos livros **(fora dos slides)**, região crítica é o **trecho de código que acessa um recurso compartilhado** e por isso exige exclusão mútua: só um processo por vez pode estar nele. Quem garante isso são semáforos, mutex, monitores ou o próprio S.O.
- **Sem exclusão mútua, o que acontece na região crítica é condição de corrida** (*race condition*): o resultado depende da ordem em que os processos executam. **Não é deadlock.**

Deadlock (impasse): um conjunto de processos em que **cada um espera um recurso que está com outro processo do mesmo conjunto**. É uma espera circular, e ninguém avança.

As 4 condições necessárias, de Coffman (fora dos slides):

1. **Exclusão mútua:** o recurso só pode ser usado por um processo de cada vez.
2. **Posse e espera** (*hold and wait*): o processo segura um recurso e espera outro.
3. **Não preempção de recursos:** um recurso não pode ser tomado de quem o detém.
4. **Espera circular.**

Como evitar deadlock: quebre qualquer uma das quatro condições.

- **Pré-alocação**: o processo só começa com **todos** os recursos. Isso quebra a posse e espera. **Custo:** recursos ficam presos e ociosos, o **throughput cai** e os processos que pedem muitos recursos podem sofrer starvation.
- **Ordenação global dos recursos**: os pedidos são feitos sempre em ordem crescente. Isso quebra a espera circular.
- **Preempção de recursos**, ou o processo libera tudo o que tem quando um pedido é negado.
- **Algoritmo do banqueiro**: só concede o recurso se o estado continuar seguro.
- **Timeout**, e detecção seguida de recuperação.

Starvation (inanição):

- O processo **espera indefinidamente** porque outros sempre passam na frente (por exemplo, o de baixa prioridade num escalonamento por prioridade).
- **Diferença para o deadlock:** no deadlock **ninguém** avança; na starvation **o sistema avança**, mas aquele processo não.
- **Como evitar: round-robin** (todos recebem um time slice), **aging** e fila FIFO de espera por recurso.

Como pode cair

- "Round-robin é justo porque evita **deadlock**": **F**. Ele evita **starvation**. Deadlock é espera por **recursos**, não por CPU.
- "Round-robin é justo porque evita starvation": **V**.
- "Escalonamento por prioridade evita starvation": **F**. Ele **causa** starvation, se não houver aging.
- "Para maximizar o throughput **e** evitar deadlock, o processo só fica pronto com todos os recursos": **F**. Isso evita deadlock, mas **reduz** o throughput. Uma frase "X e Y" é falsa se uma das partes for falsa.
- "Para minimizar o response time, o escalonador deve ser **não preemptivo**": **F**. Sem preempção, um processo longo monopoliza a CPU.

- "Na região crítica pode ocorrer deadlock": **F**. O problema na região crítica é a **condição de corrida**.
- "Com uma CPU, dois processos podem estar executando ao mesmo tempo": **F**.
- "O contexto do processo é salvo no PCB": **V**.
- "Deadlock é uma situação de contenda em que ninguém vence": **V** (rodapé do slide).

13. Sistema de arquivos (8-1 s11–s12)

- **O S.O. é responsável pela gerência de arquivos.**
- **A E/S pode ser feita byte a byte sob controle da CPU**, com laço de espera ocupado, que é **antigo e não é mais usado**.
- **Pode haver processadores dedicados à E/S**, o que **requer DMA**. As operações de um **canal** de E/S são definidas num **programa para o canal**.
- **O S.O. intermedeia o acesso** ao sistema de arquivos:
  - **traduz as solicitações** do programa em acessos às **controladoras de disco**;
  - **mantém a estrutura e a localização** dos arquivos;
  - usa **buffers** para agilizar o acesso.
- **Double buffering:** permite **processar um bloco enquanto o próximo é lido** (paralelismo).

14. Gerência de memória real (8-1 s13–s15; 8-2 s16–s19)

14.1 Princípios (8-1 s13–s14)

- A memória real é **compartilhada** entre os processos, e a memória de cada processo **precisa ser protegida** dos outros. A proteção depende **do S.O. e do suporte de hardware**.
- **Blocos de tamanho fixo** (poucas palavras): **sem fragmentação** externa.
- **Blocos de tamanho variável:** ordenados de forma decrescente, crescente ou por endereço. **Exigem técnicas contra a fragmentação**: **pior ajuste, melhor ajuste ou primeiro ajuste**. O **primeiro ajuste permite o colapso** (fusão) de blocos livres vizinhos.
- **Combater a fragmentação exige relocação** (compactação = mover os programas).
- **A memória virtual pode ser maior que a memória física**: os programas usam um espaço lógico maior que o físico. Isso **dificulta a proteção** do código.

- **A alocação contígua pode ser ineficiente** se partes do programa nunca são executadas. **A tendência atual é alocar por partes, sob demanda**, o que **facilita a proteção** contra acessos indevidos.

Fragmentação (fora dos slides, mas cobrada):

- **Interna:** espaço desperdiçado **dentro** de um bloco alocado, que é maior do que o necessário. Acontece com blocos **fixos**, como a última página ou uma partição fixa grande demais.
- **Externa:** espaço livre **total suficiente**, mas espalhado em **buracos não contíguos** entre os blocos. Acontece com blocos **variáveis**, como partições variáveis ou segmentos.

Estratégias de ajuste:

- **Primeiro ajuste** (*first fit*): o primeiro buraco que serve.
- **Melhor ajuste** (*best fit*): o menor buraco que serve; deixa sobras minúsculas.
- **Pior ajuste** (*worst fit*): o maior buraco; deixa sobras grandes, que ainda podem ser úteis.

14.2 A tabela-chave (8-1 s15 / 8-2 s15): decore

	Alocação contígua	**Alocação não contígua**
Monoprogramação	código absoluto; código relocável (estático)	**overlay**
Multiprogramação	**partições**; **swapping**; **relocação dinâmica (registrador de base)**	**paginação**; **segmentação**; **paginação-segmentação**

E a divisão por título dos slides:

- **Memória REAL:** overlay, partições, swapping, registrador de base.
- **Memória VIRTUAL:** paginação, segmentação, segmentação-paginação.

> No rodapé: "memória endereçável × memória associativa" (seção 18).

14.3 Overlay (8-2 s16)

- Usado quando o **espaço lógico do usuário é maior que a memória física**.
- O programa é dividido em uma **fase raiz**, que fica **fixa na memória**, e **fases móveis**, organizadas numa árvore (F1, F2, F3; F11...F1n; F31...F3m). Fases do mesmo nível **compartilham a mesma área** e se sobrepõem (*overlay*).
- **Requisito de memória: raiz + o caminho mais "pesado" da árvore** (seção 18).

14.4 Partições (8-2 s17)

- **Fixas:** a memória é dividida em partições de tamanho fixo, com uma **fila por partição**. Causam **fragmentação interna**, porque o programa é menor que a partição.
- **Variáveis:** cada programa recebe exatamente o que precisa. Causam **fragmentação externa** e exigem os ajustes da seção 14.1 e compactação.
- O rodapé do slide diz: **"Problema de fragmentação da memória."**

14.5 Swapping (8-2 s18)

- **Troca processos via interrupção (time slice)**: tira um processo inteiro da memória e o leva ao disco, e traz outro.
- Usa **escalonamento por prioridade**.
- **Tempos de remoção e carga elevados** (milissegundos).
- Há uma **fila de prontos** e uma **fila de bloqueados**. **Os processos na fila de bloqueados esperam a liberação de algum recurso.**
- É a gerência real típica do **time sharing**.

14.6 Registrador de base: relocação dinâmica (8-2 s19)

- **A relocação é feita durante a execução.**
- **Endereço real = endereço lógico + RegBase.**
- Por desempenho, a soma é feita em hardware pelo **DAT** (*Dynamic Address Translator*).
- **Facilita a relocação do código na memória**: para mover o programa, basta mudar o RegBase. Por isso ajuda no swapping e na compactação.
- É a **forma mais simples de relocação dinâmica**. O **Intel 8086** usa algo parecido (segmento × 16 + offset). O **80386 tem MMU**.



Relocação estática × dinâmica:

	Estática	Dinâmica
Quando ocorre	antes da execução (tradução, ligação, carga)	**durante** a execução, a cada acesso
Quem faz	ligador / carregador de relocação	hardware: DAT/MMU com RegBase ou tabela de páginas

	Estática	Dinâmica
Depois de carregar, pode mover o programa?	**não** (seria preciso relocar de novo)	**sim** (basta trocar o RegBase ou a tabela)

Como pode cair

- "Paginação é uma técnica de gerência de memória **real**": **F**. Nos slides, paginação é memória **virtual**.
- "Swapping é alocação não contígua": **F**. Swapping é **contígua**.
- "O overlay é usado em multiprogramação": **F**. Na tabela, ele está em **monoprogramação**.
- "Com o registrador de base, a relocação é estática": **F**. É **dinâmica**.
- "Endereço real = endereço lógico × RegBase": **F**. É uma **soma**.
- "Combater a fragmentação dispensa relocação": **F**. **Exige** relocação.
- "Blocos de tamanho variável não têm fragmentação": **F**.
- "Melhor ajuste permite o colapso de blocos": **F**. O slide atribui o colapso ao **primeiro ajuste**.

15. Gerência de memória virtual (8-2 s20–s23)

15.1 Paginação (8-2 s20–s21)

- É uma **abordagem de relocação dinâmica.**
- **A memória é dividida em páginas de tamanho fixo** (alguns poucos KB, potência de 2).
- **Na carga, só a página que contém o entry point é carregada.** As outras vêm **sob demanda**: quando o processo acessa uma página que não está na memória, ocorre um **page fault**.
- **A conversão do endereço virtual para o real é feita pelo DAT**, usando uma **tabela de mapeamento de páginas**. Cada página do programa tem uma entrada, com o **frame físico** e bits de controle.
- **Quando não há espaço**, um algoritmo escolhe a página a remover: **LRU** (*least recently used*). O **bit de referência é zerado de tempos em tempos**, e assim o S.O. sabe quais páginas foram usadas recentemente.
- **Facilita a proteção** contra acesso indevido ao código (seção 18).

Mapeamento lógico → real (diagrama de 8-2 s21):

```
Endereço lógico = [ nº da página (e-d) | deslocamento d ]

  RegBase (início da tabela do processo) + nº da página → entrada na
  tabela de mapeamento de páginas: [ r | s | p ]  (p = frame real; os demais são bits de

Endereço real = [ p | d ]   ← CONCATENAÇÃO de p com d
```

- O **deslocamento d passa sem alteração**. Só o número da página é traduzido.
- O operador é a **concatenação**, e não a soma, porque as páginas têm tamanho **fixo, potência de 2**, e começam em endereços múltiplos do tamanho da página. Os bits baixos do início do frame são zero, então concatenar dá o mesmo resultado que somar, mas é mais rápido: não há propagação de "vai-um".

Exemplo resolvido (estilo discursiva):

- Páginas de **4 KB = 2^{12} → d tem 12 bits**.
- Endereço lógico **0x3A7F** → página = 0x3A7F >> 12 = **3**; d = 0x3A7F & 0xFFF = **0xA7F**.
- Na tabela, a página 3 está mapeada no frame **0x15**.
- **Endereço real** = 0x15 ‖ 0xA7F = **0x15A7F**. Confira: 0x15 × 0x1000 + 0xA7F = 0x15000 + 0xA7F.
- Se a entrada da página 3 indicasse "não está na memória", ocorreria um **page fault**: o processo é bloqueado, o S.O. lê a página do disco (escolhendo uma vítima pelo LRU, se não houver frame livre), atualiza a tabela, e o processo volta para os prontos. É o roteiro da seção 3.1.

O mesmo exemplo em decimal: páginas de **1024**; endereço lógico **2100** → página = 2100 div 1024 = **2**, d = 2100 mod 1024 = **52**. A página 2 está no frame 7, então o real é 7 × 1024 + 52 = **7220**.

15.2 Segmentação (8-2 s22)

- É parecida com a paginação, mas **os blocos têm tamanho variável**, e o **tamanho está associado à lógica do programa**: um segmento de código, um de dados, um de pilha, uma rotina.
- **O endereço real é obtido por SOMA** (base do segmento + deslocamento). **O operador de concatenação não pode ser usado**, porque o segmento pode começar em qualquer endereço.
- **O custo operacional é maior** que o da paginação.
- **Facilita muito a proteção do código**: cada segmento é uma unidade lógica com seus próprios direitos, por exemplo código só para execução.

15.3 Segmentação-paginação (8-2 s23)

- Tenta **aproveitar as vantagens dos dois métodos**.

- **O segmento tem tamanho variável, mas é múltiplo do tamanho da página.**
- **O endereço lógico tem três níveis: segmento, página e deslocamento.**
- **Exige uma tabela de segmentos e uma tabela de páginas para cada segmento.**
- Depois de determinada a página, **o endereço real é obtido por concatenação** da página real com o deslocamento.
- É **largamente utilizado hoje**.

15.4 Comparação final

	Paginação	Segmentação	Segmentação-paginação
Bloco	**fixo**	**variável** (lógico)	variável, **múltiplo da página**
Fragmentação **interna**	**sim** (última página)	**mínima**	sim (pouca)
Fragmentação **externa**	**não**	**sim**	não
Operador	**concatenação**	**soma**	concatenação (no final)
Tabelas	uma de páginas	uma de segmentos	de segmentos + de páginas por segmento
Custo	menor	**maior**	intermediário
Proteção	facilita	**facilita muito**	facilita

Como pode cair

- "Paginação minimiza a fragmentação interna e favorece a externa": **F**. Isso descreve a **segmentação**. A paginação tem **interna** e **não tem externa**.
- "Segmentação minimiza a fragmentação interna, mas favorece a proteção e a fragmentação externa": **V**.
- "Na segmentação, o endereço real é obtido por concatenação": **F**. É **soma**.
- "Na paginação, o endereço real é obtido somando p e d": **F**. Pelo slide, o operador é a **concatenação**.
- "Na carga, todas as páginas são carregadas": **F**. Só a do **entry point**.
- "A segmentação tem custo menor que a paginação": **F**.
- "Na segmentação-paginação, o endereço lógico tem dois níveis": **F**. Tem **três**.
- "Os segmentos têm tamanho fixo": **F**. Também é F dizer que, na segmentação-paginação, o segmento é de tamanho qualquer: ele é **múltiplo da página**.

- "O LRU remove a página mais recentemente usada": **F**. Remove a **menos** recentemente usada.

16. Segurança e estruturação do S.O. (8-2 s24–s28)

16.1 Segurança e recuperação de erros

- Os usuários devem ser **protegidos de ações não autorizadas** de outros usuários.
- **O controle de acesso** costuma se basear numa **matriz de acesso** (sujeitos × objetos → direitos). Na prática, ela é guardada de duas formas:
  - **lista de autorizações** (ACL: para cada objeto, quem pode o quê);
  - **lista de capacidades** (para cada sujeito, o que ele pode).
- **Depende de mecanismos de autenticação.**
- Os privilégios são separados em **proprietário, grupo e demais usuários**.

16.2 Opções de estruturação (8-2 s25)

- **Estrutura hierárquica (camadas):**
  - dá um **excelente nível de abstração**;
  - **cada nível usa os serviços dos níveis inferiores**;
  - **o nível 0 é o kernel**: tratamento de interrupções, escalonamento e outros serviços essenciais.
- **Máquinas virtuais:**
  - criam a **ilusão de máquinas reais separadas**;
  - **diferentes S.O. rodam simultaneamente** no mesmo computador (IBM VM);
  - as rotinas de baixo nível do S.O. convidado interagem com a **máquina virtual**, não com a real;
  - **a máquina virtual simula o efeito das instruções privilegiadas** executadas pelos S.O. convidados.
- **Multiprocessados:**
  - vários processadores (por exemplo, o IBM /390 com 64 processadores, rodando z/OS e Linux);
  - **fortemente acoplados**: os processadores **compartilham a memória**;
  - **fracamente acoplados**: cada processador tem **memória local** própria.
- **Distribuídos:**
  - podem ter **supervisores separados**, ser **mestre-escravo** ou **simétricos**;
  - **S.O. de rede não torna a rede transparente**; **S.O. distribuído torna a rede transparente**.

- **Orientados a objetos:**
  - o S.O. é uma **coleção de objetos**, implementados por processos **servidores (gerentes de objetos)**, que criam e destroem os objetos;
  - **o núcleo faz só o simples**: comunicação entre processos e escalonamento;
  - **a maior parte das atividades é feita por objetos, não pelo kernel.** O exemplo do slide é a arquitetura do Windows NT.

Como pode cair

- "Na estrutura hierárquica, um nível usa os serviços dos níveis **superiores**": **F**. Usa os dos níveis **inferiores**.
- "O nível mais alto é o kernel": **F**. O kernel é o **nível 0**.
- "Fortemente acoplado = cada processador tem sua memória": **F**. Isso é o **fracamente acoplado**.
- "Em S.O. orientado a objetos, o kernel faz a maior parte do trabalho": **F**.
- "Na máquina virtual, os S.O. convidados executam diretamente as instruções privilegiadas na máquina real": **F**. A VM **simula** essas instruções.

17. Boot do Windows XP (9-boot)

Visão geral: o boot **começa quando o computador é ligado** e **termina com o logon do usuário**. Ele usa uma **cadeia de carregadores**, cada um carregando o seguinte:

```
ROM BIOS (código absoluto, FFFF:0000) → MBR (setor 1 do disco) → boot record da partiçã
→ NTLDR → NTOSKRNL.EXE + HAL.DLL → SMSS → Winlogon/LSASS → logon
```

17.1 As 5 fases

1. **Sequência de pré-boot:** testes de hardware e carga do bootstrap.
2. **Sequência de boot:** detecção do hardware, seleção do S.O. e do perfil.
3. **Carga do kernel:** carga dos drivers selecionados e do kernel.
4. **Iniciação do kernel:** inicia o kernel e os drivers e carrega a GUI.
5. **Logon.**

Arquivos e fases:

Arquivo	Onde fica	Fase
NTLDR	raiz C:\	pré-boot e boot
BOOT.INI, bootsect.dos, ntdetect.com	raiz C:\	boot
ntoskrnl.exe, hal.dll	systemroot\System32	carga do kernel
system	systemroot\System32\Config	iniciação do kernel
drivers *.sys	systemroot\System32\Drivers	iniciação do kernel

(*systemroot* costuma ser C:\WINDOWS. **HAL** = *Hardware Abstraction Level/Layer*.)

17.2 Pré-boot

1. **Ligação:** se os níveis de tensão e corrente estão aceitáveis, a fonte envia o sinal **Power Good** à CPU. O **timer recebe o Power Good e para de enviar o reset** para a CPU.
2. **A CPU executa a ROM BIOS:** em **FFFF:0000** há um **jmp** para o código de startup. Esta é a ligação com a seção 4: é **código absoluto**.
3. **A BIOS testa o hardware básico.** Erros são reportados por **beep codes**, porque o vídeo ainda não foi iniciado.
4. **Detecção de dispositivos plug and play:** a BIOS procura **extensões de BIOS**, sendo a mais comum a do **vídeo**. Ela varre de **C000:0000 a C780:0000**. Se der erro, aparece a mensagem "**XXXX ROM Error**", onde XXXX é o segmento do módulo que falhou.
5. **Cold × warm start:** a BIOS lê a word em **0000:0472**.
   - Se vale **1234h**, é **warm start**, e o **POST** é feito **sem o teste de memória**.
   - Senão, é **cold start**, e o **POST** é completo: vídeo, identificação da BIOS e teste de memória.
   - Erro **não fatal**: aparece mensagem na tela e o boot continua. Erro **fatal**: beep codes e a carga para.
   - POST = *Power On Self Test*.
6. **CMOS:** 64 bytes mantidos por **bateria**, com as configurações. A mais importante para o boot é a **ordem dos dispositivos de boot** (disquete, CD-ROM, disco fixo, USB).
7. **No disco fixo, a BIOS lê o MBR** (*Master Boot Record*):
   - é o **primeiro setor (512 bytes)**: **cilindro 0, cabeça 0, setor 1**;
   - é carregado em **0000:7C00**;
   - a BIOS testa a **assinatura 55AAh nos dois últimos bytes**. Se o MBR ou a assinatura não forem encontrados: "**NO ROM BASIC – SYSTEM HALTED**";
   - o MBR tem **duas partes**: a **tabela de partições** e o **código do carregador da partição** (*boot loader*).

8. **O boot loader** examina a tabela de partições, acha a **partição ativa** e lê o **registro de boot** dela, que fica no setor 1 da partição e tem 512 bytes. Esse registro contém as características da partição (bytes por setor, setores por cluster) e o código que carrega o **NTLDR**.
    - Para ter **vários S.O.**, troca-se o carregador original por um que deixa o usuário escolher.
    - **Em disquete:** a BIOS busca o registro de boot no primeiro setor do disquete.
    - **A partir daqui, o boot é controlado pelo NTLDR.**

17.3 Sequência de boot (controlada pelo NTLDR)

1. **Carga inicial:** o NTLDR muda o processador do **modo real para o modo protegido**, com **endereços de 32 bits e memória virtual paginada**. Em seguida carrega um **mini sistema de arquivos** (FAT16, FAT32 ou NTFS).
2. **Seleção do S.O.:** o NTLDR lê o **BOOT.INI**.
    - Se houver mais de um S.O., mostra um menu e espera; se o usuário não escolher, carrega o padrão.
    - **Sem BOOT.INI**, tenta carregar o Windows a partir da **primeira partição do primeiro disco**.
    - **F8** mostra opções como **Safe Mode** e **Last Known Good Configuration** (a última combinação de drivers e configurações que carregou com sucesso).
3. **Detecção do hardware:** o **NTDETECT.COM** coleta a lista de hardware e a entrega ao NTLDR, que a grava no Registry em **HKEY_LOCAL_MACHINE\HARDWARE**.
4. **Perfil de hardware:** se houver vários perfis, aparece um menu. Um perfil pode fazer o Windows **ignorar componentes** e não carregar seus drivers. Com um único perfil, o passo é pulado.

17.4 Carga do kernel

- O **NTOSKRNL.EXE é carregado, mas não iniciado.** O **HAL.DLL** também é carregado. Na tela aparecem retângulos brancos.
- A chave **HKLM\SYSTEM** é carregada de *Systemroot\System32\Config\System*, e um **control set** é escolhido. O control set define quais drivers e serviços sobem automaticamente.
- Os drivers com **SERVICE_BOOT_START** são carregados: são os de **baixo nível**, como os de disco. Aparece **um ponto por driver**.

17.5 Iniciação do kernel (2 fases)

- **Fase 0:**
    - inicia o microkernel e os subsistemas executivos, e aparece a tela gráfica com a barra de progresso;
    - **as interrupções ficam desabilitadas** enquanto o **HAL prepara o controlador de interrupções**, e depois são **habilitadas**;

    - sobem o Memory Manager, o Object Manager, o Security Reference Monitor e o Process Manager.
- **Fase 1:**
    - inicia os processadores adicionais;
    - reinicia os executivos;
    - **cria a chave HKLM\HARDWARE**;
    - faz uma **cópia de segurança do control set**;
    - **inicia os drivers** com o I/O Manager e os serviços automáticos;
    - lança o **SMSS** (*Session Manager Subsystem*), que cria o ambiente user-mode e carrega o **win32k.sys** (o subsistema gráfico Win32).

17.6 Logon

O Win32 inicia o **Winlogon.exe**, que inicia o **LSASS.exe** (LSA, *Local Security Authority*) e mostra a janela de logon. **O boot só é considerado OK quando o usuário faz o logon.**

17.7 BOOT.INI e caminhos ARC

```
[boot loader]
timeout=30
default=multi(0)disk(0)rdisk(1)partition(2)\WXPPRO
[operating systems]
multi(0)disk(0)rdisk(1)partition(2)\WXPPRO="MS Windows XP Professional" /fastdetect
C:\="MS Windows 98"
```

O Windows 98 e o ME usam caminho **convencional**, não ARC (*Advanced RISC Computing*).

Partes do caminho ARC:

- **multi(n) / scsi(n):**
    - **scsi** se o disco está numa controladora SCSI;
    - **multi** se o disco é IDE, EIDE, ESDI, ATA ou SATA, ou numa SCSI **sem BIOS própria**;
    - n é a **ordem do adaptador**, começando em 0.
- **disk(n):** com scsi, é o **ID do disco SCSI**; com multi, é **sempre disk(0)**.
- **rdisk(n):** o número do disco **dentro da controladora**, **começando em 0**. É ignorado em SCSI.
- **partition(n):** o número da partição, **começando em 1**.
- Depois vêm a **pasta do S.O.**, o **="descrição do menu"** e as **chaves**.

Exemplo: `multi(1)disk(0)rdisk(0)partition(2)` é a **2ª partição do 1º disco da 2ª controladora**.

Chaves mais usadas:

Chave	Efeito
/basevideo	força o driver VGA padrão
/bootlog	grava o Ntbtlog.txt na raiz
/fastdetect	o NTDETECT não detecta dispositivos nas portas seriais (é o padrão no XP); `/fastdetect:comN` limita a portas específicas
/HAL=arquivo	escolhe o HAL (por exemplo, halmps.dll para multiprocessador)
/kernel=arquivo	escolhe o kernel (por exemplo, ntkrnlmp.exe, com mais depuração)
/maxmem=MB	limita a RAM (útil se um chip estiver com defeito)
/noguiboot	desliga o bitmap com a barra de progresso
/numproc=n	limita o número de processadores
/pcilock	o S.O. não atribui E/S e IRQs aos dispositivos PCI; quem configura é a BIOS
/sos	mostra os nomes dos drivers em vez dos pontos

Como pode cair

- "A ROM BIOS começa em 0000:7C00": **F**. A BIOS começa em **FFFF:0000**; o MBR é que é carregado em 0000:7C00.
- "A assinatura válida do MBR é 1234h": **F**. A assinatura é **55AAh**; 1234h em 0000:0472 é o indicador de **warm start**.
- "No warm start, o POST faz o teste de memória": **F**. O warm start pula o teste de memória.
- "O NTLDR muda do modo protegido para o real": **F**. Muda **do real para o protegido**.
- "Quem detecta o hardware é o NTLDR": **F**. Quem detecta é o **NTDETECT.COM**; o NTLDR só grava a lista no Registry.
- "rdisk começa em 1" ou "partition começa em 0": **F**. É o contrário: **rdisk começa em 0** e **partition começa em 1**.
- "Na carga do kernel, o NTOSKRNL é iniciado": **F**. Ele é **carregado, mas não iniciado**; a iniciação é a fase seguinte.
- "O boot termina quando a GUI aparece": **F**. Termina com o **logon**.
- "Erros do POST aparecem na tela antes de o vídeo ser iniciado": **F**. Eles são reportados por **beep codes**.

18. Perguntas dos rodapés dos slides (respondidas)

O professor coloca perguntas no rodapé dos slides, e elas podem virar discursiva ou item V/F.

1. Qual o efeito de uma instrução de desvio incondicional? (7-1 s7)

- O PC recebe o **endereço de destino** em vez de ser apenas incrementado. Isso **quebra o processamento sequencial**.
- Consequências:
  - as instruções que já tinham sido buscadas antecipadamente no **pipeline** são **descartadas** (*flush*), e o pipeline precisa ser reabastecido;
  - a **localidade espacial é quebrada**, o que pode gerar **falha de cache** ou até **page fault** no destino.

2. Quantos buffers são recomendáveis? Por quê? (7-1 s8)

- **Pelo menos dois** (*double buffering*): enquanto a CPU ou o S.O. processa um buffer, o processador de E/S (DMA ou canal) enche ou esvazia o outro. Com isso, processamento e E/S ficam **em paralelo**.
- Com um só buffer, a CPU espera cada leitura física.
- Mais de dois ajudam quando as velocidades de produção e consumo variam.

3. Qual o mecanismo de reconhecimento da fonte de uma interrupção? (7-1 s9; resposta em 8-1 s8)

- A **cadeia de reconhecimento** (*daisy chain*): a CPU propaga o **ACK** pelas fontes em **ordem decrescente de prioridade**. A primeira fonte que quer interromper **segura o ACK** e coloca sua **identificação no barramento de dados**.
- A CPU usa essa identificação como **índice no vetor de interrupções**, que aponta para a rotina de tratamento.

4. A ROM "funciona sem energia devido a utilizar pontes de silício" (7-1 s13).

- É só uma observação: a ROM é **não volátil**. Por isso o código absoluto de boot fica nela.

5. Por que a referência futura é mais complexa? Como tratar? Qual o número de passos característico hoje? (7-1 s18)

- **Por que é complexa:** no momento em que a instrução é gerada, o símbolo **ainda não tem endereço**, porque o PC do montador ainda não chegou à definição dele.
- **Como tratar:**

    - **Em dois passos:** o passo 1 só percorre o fonte calculando o PC e montando a **tabela de símbolos completa**; o passo 2 gera o código, já com todos os endereços conhecidos.
    - **Em passo único:** gera a instrução com o campo vazio e guarda a posição numa **lista de pendências** do símbolo (por exemplo, uma lista encadeada que passa pelos próprios campos vazios). Quando o símbolo é definido, percorre a lista e **preenche os campos** (*backpatching*).
- **Número de passos hoje: dois ou mais passos.**
    - **Por quê:** memória e tempo de processamento não são mais um problema, e separar as tarefas é mais simples, mais modular e permite **otimização** (o "passo n").
    - Esta resposta é a leitura mais comum; o slide só faz a pergunta.

6. Qual o problema com o esquema do carregador de relocação? (7-1 / 7-2 s49)

- Ele exige um **bloco contíguo** do tamanho do **programa inteiro**. Por isso:
    i. sofre **fragmentação externa**: pode haver memória livre suficiente, mas espalhada;
    ii. um programa **maior que o maior bloco livre não roda**;
    iii. o programa é **carregado inteiro**, inclusive as partes que nunca executam (desperdício);
    iv. a relocação é **feita uma vez, na carga**, então o programa **não pode ser movido depois**. Isso atrapalha a compactação e o swapping para outra posição.
- Essas limitações motivam o **registrador de base** (relocação dinâmica), a **paginação** e o **overlay**.

7. Qual o requisito de memória do programa com overlay? (8-2 s16)

- **Tamanho da raiz + o maior tamanho de caminho da raiz até uma folha** da árvore de fases.
- **Exemplo:** raiz 20 KB; F1 = 10 com filhos F11 = 15 e F12 = 5; F2 = 30; F3 = 8 com filhos F31 = 12 e F32 = 25.
    - Os caminhos são: F1+F11 = 25, F1+F12 = 15, F2 = 30, F3+F31 = 20 e F3+F32 = **33**.
    - O requisito é 20 + 33 = **53 KB**, e não a soma de tudo (125 KB).

8. Por que a paginação facilita a proteção contra acesso indevido ao código? (8-2 s20)

- Todo acesso passa pela **tabela de páginas do próprio processo**, então ele **só consegue alcançar os frames que estão mapeados nela**. As páginas de outros processos simplesmente não existem no seu espaço de endereços.
- Além disso, cada entrada pode ter **bits de proteção** (por exemplo, página de código só leitura/execução). O DAT verifica esses bits a cada acesso e gera uma interrupção se houver violação.

9. Qual operador aplicar a p e d para gerar o endereço real? Por quê? (8-2 s21)

- A **concatenação** (p ‖ d), porque as páginas têm **tamanho fixo e potência de 2** e cada frame começa num endereço múltiplo do tamanho da página. Os bits baixos do início do frame são zero, então concatenar equivale a somar, mas é **mais rápido** em hardware (não há carry).
- Na **segmentação** isso não é possível: o segmento começa em qualquer endereço, então é preciso **somar**.

10. Memória endereçável × memória associativa (8-1 s15 / 8-2 s15)

- **Memória endereçável** (RAM comum): o acesso é feito **pelo endereço**.
- **Memória associativa** (CAM): o acesso é feito **pelo conteúdo ou chave**, comparando **todas as posições em paralelo**.
- Na gerência de memória, a memória associativa é usada como **cache da tabela de páginas** (a **TLB**): ela guarda os pares página → frame mais usados e evita o acesso extra à tabela na memória, a cada tradução.

11. "Qual o problema com esse esquema?" e "Por que a relocação viabiliza bibliotecas?" (7-1 s14 e s23)

- Sem relocação, uma biblioteca objeto teria **endereços fixos** e só poderia ser usada se ficasse sempre na mesma posição, o que gera conflito entre bibliotecas.
- Com relocação, a mesma biblioteca objeto é **ligada em qualquer posição** de qualquer programa, sem recompilar.

19. Mapa mental: quem faz o quê

Pergunta	Resposta
Quem traduz Assembly em objeto?	**Montador** → objeto **relocável**
Quem resolve as referências externas?	**Ligador**
O que o ligador gera?	código de máquina **relocável e executável**
Quem põe o programa na memória e ajusta os endereços?	**Carregador de relocação** (na carga)
Quem carrega o S.O. no boot?	**Carregador absoluto** (ROM, IPL)
Quem coloca o entry point no PC?	o **carregador**, ao transferir o controle

Pergunta	Resposta
Quem resolve as funções de DLL em execução?	o **carregador**, com a tabela **fixup**
Quem move pronto → executando?	**Escalonador**
Quem move bloqueado → pronto?	**Interrupção** (controlador de interrupções)
Quem traduz o endereço lógico em real em execução?	**DAT/MMU** (com RegBase ou tabela de páginas)
Quem escolhe a página vítima?	**LRU** (com o bit de referência)
Quem detecta o hardware no boot do XP?	**NTDETECT.COM**
Quem lê o BOOT.INI?	**NTLDR**
Quem faz o logon?	**Winlogon → LSASS**

Relocação: todos os "quandos"

- **Estática** (tradução, ligação ou carga): montador (marca), ligador (k = tamanhos), carregador de relocação (+α).
- **Dinâmica** (em execução): registrador de base (soma), paginação (concatenação), segmentação (soma).
- **Ligação dinâmica** (em execução, na primeira chamada): DLL / .so.

20. Como atacar um item V/F inédito

1. **Procure a palavra trocada.** Pares clássicos que o professor inverte:

Uma palavra	↔	A outra
relocável	↔	absoluto / executável
volátil	↔	não volátil
interna	↔	externa (fragmentação)
páginas	↔	segmentos
concatenação	↔	soma

Uma palavra	↔	A outra
starvation	↔	deadlock
batch	↔	time sharing
CPU intensivo	↔	E/S intensivo
preemptivo	↔	não preemptivo
real	↔	virtual (gerência de memória)
contígua	↔	não contígua
estática	↔	dinâmica
usuário → supervisor	↔	supervisor → usuário
inferiores	↔	superiores (camadas)
crescente	↔	decrescente (prioridade no daisy chain)
montador	↔	ligador

2. **Desconfie de absolutos:** "apenas", "sempre", "nunca", "não é mais usado", "deve ser", "o melhor". Só são V se o slide disser exatamente isso.
3. **"X e Y" e "X, pois Y":** basta uma parte falsa, ou uma justificativa errada, para o item ser **F**. Exemplos: "evita deadlock **e** maximiza o throughput"; "relocável, **como o bytecode Java**".
4. **Lista incompleta × processo resumido:**
   - Se a frase se apresenta como **a lista que define algo** ("é caracterizada por", "é composta por", "são dos tipos") e **falta um item do slide** → tende a **F**. Justifique citando o item que falta.
   - Se só **descreve um processo** e **tudo o que afirma está correto**, mesmo omitindo detalhes → tende a **V**.
5. **Exemplo errado num conceito certo** → **F**.
6. **Justificativa do F:** uma frase só, que diga **o que está errado** e **qual é o certo**. Por exemplo: "O montador gera código **relocável**, não absoluto." Evite justificativas vagas, como "depende do sistema".
7. **NS:** como dois erros anulam um acerto, marcar uma resposta tem valor esperado positivo sempre que você tem mais de cerca de 1/3 de confiança. Use NS só quando estiver totalmente perdido.